Working Agreement

MATT SYP
Consulting



Table Of Contents

Purpose and Overview....3
Instructions and Recommendations....3
Methodology....4
Meetings and Ceremonies....6
Tools and Workflows....8
Documentation....11
Code Reviews....13
On Call / Support....15
Additional Topics....18
Topic Template....20

Purpose and Overview

This Working Agreement template helps teams establish how they work together. It is strictly about the tools, patterns and practices the team agrees to try, rather than the specific projects being worked on.

Use this framework when forming a new team, refining existing processes, onboarding new members or any time your team is experiencing change or friction. As a living document, it should be continuously updated to reflect your team's evolving practices, ensuring that everyone has a say in how the team operates and comes to a shared agreement.

The template covers key operational areas including methodology, ceremonies, tools and documentation, with usage examples provided to help guide the discussion.

Instructions and Recommendations

Most topics have 4 sections: Summary, Current Process, Discussion Notes, and Agreements.

Summary

- Describes the topic and provides information for the discussion including Key Questions & Discussion Prompts to guide the team's conversation and ensure you cover the most important details.

Current Process

- Documents the current process for that topic.
- Can be pre-filled in prior to the discussion or could be filled in during the discussion.
- Be as detailed as possible so you can reference the changes being made easily.

Discussion Notes

- Add all relevant notes for points made during the discussion. This should be your living document section that provides data around the agreements that were made.

Agreements

- All agreements made by the team need to be in this section.
- The agreements should be as detailed as possible.
- This template now includes Example Agreements (based on real-world examples) to show the level of detail and clarity the team should aim for.
- Not all team members need to agree to document the agreements; allow the team to "disagree and commit" to trying. Any disagreements should be documented in the Discussion Notes section.

Methodology

Summary

Scrum vs Kanban. If possible, discuss both options and understand what is best for the team. I recommend choosing the methodology that works best for the team as the base and combining ceremonies and practices from both to customize it to fit the needs of the team. If your company doesn't allow a choice, use this section to understand how you can make the best out of the methodology you must use.

A good blog to read before this discussion

- https://www.teamwork.com/blog/kanban-vs-scrum/
- https://www.usemotion.com/blog/agile-ceremonies
- https://businessmap.io/agile/project-management/ceremonies

Key Questions & Discussion Prompts:

- What is our primary motivation for this discussion? (e.g., Do we want better flow? Fewer ceremonies? More flexibility? Better engineer engagement?)
- What parts of our *current* methodology create friction? What parts provide high value?
- If we move from Scrum to Kanban: How will we ensure we still have clear, "demo-able" goals and predictable delivery commitments for our product managers and stakeholders?
- If we move from Kanban to Scrum: How will we handle emergent, high-priority work that can't wait for a new sprint?
- What is the practical "cost" of switching? (e.g., Creating a new Jira board, migrating tickets, changing team habits?)
- If we *cannot* change our high-level methodology (e.g., "company uses Scrum"), what specific ceremonies or practices can we change to improve our team's daily life?

Current Process

Seems like 2 week sprints using Scrum, but the board shows backlog work in progress. No real sprint management so probably more towards Kanban.

Discussion Notes

Agreements

Kanban starting in November

All agreements made by the team need to be in this section. The agreements should be as detailed as possible.

Example Agreement: Moving from Scrum to Kanban

> **Our Methodology**: We are moving to Kanban, starting in 2026.
>
> **Transition Plan**: Sprints will continue as-is to finish the year. [Team Member] will create the new Kanban board and get everything ready for the 2026 start.
>
> **Role Changes**: We will no longer have a rotating "Sprint Captain." Instead, [Team Member] will be the standing facilitator for stand-ups and retros with the team picking up ad-hoc if they are out.

Meetings and Ceremonies

Summary

Every team has recurring meetings and ceremonies. What works for one team may not work for another. The goal of this section is to discuss if the current ceremonies are providing value and decide which to keep, remove, modify or add. If you are moving from one methodology to another, discuss the standard ceremonies for that methodology to see if they may bring value. I also recommend discussing a no-meeting day for engineers so they can have a day of deep focus.

Key Questions & Discussion Prompts:

Meeting Cadence:

- Which standard ceremonies (e.g., Stand-up, Refinement, Planning, Retro, Demo) will we hold, and how often?
- What is the ideal duration for each? (e.g., Grooming: 1 hour/week)

Purpose & Value:

- What is the explicit purpose of each meeting? (e.g., "First 10 minutes of grooming will be for prompts/blockers")
- How can we ensure each meeting stays focused on its purpose?

Async vs. Sync:

- Can any of our current synchronous meetings be made asynchronous?
- Example: For async stand-ups, what format will we use? What content needs to be shared daily? What time are they "due"?

Ground Rules:

- How do we handle "no meeting days"? What are the exceptions?
- What are our team's ground rules for effective meetings? (e.g., Start on time, come prepared).

Current Process

Current Ceremonies

- Every Monday
    - Pipelines PED Weekly Sync
- Every Weekday
    - Async stand up in #payper-pipeline-teams
- No meeting Thursdays
- Monthly Retro
- Every other Friday

    - social half hour
    - Playing games

Discussion Notes

Previously ran weekly workstream stand ups
HD: Seems like there is a good place for product and customer discussions, but maybe not for the tech side of things.

- Current tech time meeting is too big but used to work good
- Hard to get into specific features for your team at this point
- Maybe refinement would be a good way to do this?
- Maybe in person PR reviews or design reviews?
- Maybe a mid-week meeting would be best for this

Agreements

All agreements made by the team need to be in this section. The agreements should be as detailed as possible.

Example Agreement: Stand-ups (Async/Sync Hybrid)

Frequency: We will use async stand-ups posted in the `@team-channel` channel every day. We will also hold a synchronous (in-person) stand-up meeting daily for 15 minutes.

Async Format:

- The stand-up will consist of 3 parts: Yesterday (Y), Today (T) and Prompts/Blockers (P/B).
- All posts should be in the thread by 12pm Eastern, 9am Pacific.
- Y (Yesterday): Include ticket numbers and a brief description of what you did (e.g., "TIX-9999: Finished coding and need to complete unit tests").
- T (Today): Include ticket numbers and your plan (e.g., "TIX-9999: Complete unit tests and move into In Review").
- P/B (Prompts/Blockers): This is a trigger for conversation. If you need help, tag the relevant people (e.g., "TIX-9999: I am unable to get my environment setup for testing, `@team-eng` can someone help").

Daily Sync Meeting:

- During the daily in-person meetings, we will *only* discuss the prompts and blockers posted in the async thread. We will not read what we did yesterday or what we are doing today.

Tools and Workflows

Summary

This topic is focused on how the team organizes its work and how it should flow through the chosen tools (e.g., Jira). The goal is to create a clear, predictable, and transparent process.

Key Questions & Discussion Prompts:

Ticket Types:

- What are the different types of tickets we use, and what is the purpose of each? (e.g., Initiative, Epic, Story, Task, Spike, Bug, Sub-Task).

Ticket Requirements:

- What defines a "ready" ticket? What information must be included before an engineer can begin work?
- *Guiding Principle*: "Write tickets as if you will not execute on them".
- *Requirement Example*: "Acceptance criteria should be part of every ticket".
- How detailed should tickets be for different parts of the stack (e.g., [API], [Database], [FrontEnd])?

Story Points (if applicable):

- What do our points mean? (e.g., Complexity, effort, time?).
- What is included in an estimate (e.g., testing, DB changes, merging, deployment)?
- What does our scale look like? (e.g., 1 = 1-liner, no testing; 2 = requires testing; 8 = needs discussion to be broken down).

Board Columns / Workflow:

- What are the exact stages a ticket goes through from idea to production?
- *Example Engineering Board*: Backlog -> Up Next -> In Progress -> Blocked -> In Review -> In Testing -> Ready for Release -> Done.

Definition of Done (DoD):

- What must be true for a ticket to move out of each key column?
- *Example (for "In Progress")*: What needs to be done before it can move to "In Review"?
- *Example (for "In Review")*: What needs to be true before it can be merged?

Work-In-Progress (WIP) Limits:

- Do we want to set limits on any columns (e.g., "In Progress," "In Review") to improve flow and prevent bottlenecks?
- *Example Formula*: "In Progress" limit = [Number of Engineers] * 1.5.

Current Process

Ticket Types

- Epic
- Story
- Task
- Spike
- Bug
- Sub-task
- Security

Boards / Columns

- REP-Payer Pipelines
    - To Do
        - To Do
        - Scoping
        - Pending Approval
        - Ready for Development
    - Blocked
        - Blocked
    - In Progress
        - In Progress
    - Code Review
        - Code Review
    - Testing
        - QA
        - UAT
    - Ready For Release
        - Ready for Release
    - Done
        - Done
        - Not a Bug
        - Won't Do
        - Cannot Reproduce
        - Duplicate

Notes

- Tickets don't have a ton of info in them.
- Titles are either a short description of the feature or a short description of the task that needs to be completed.

Discussion Notes

Agreements

All agreements made by the team need to be in this section. The agreements should be as detailed as possible.

Example Agreement: Ticket Requirements & Structure

Hierarchy:

- **Epic:** A fully implemented new feature/body of work.
- **Story:** A group of tasks that together make up some functionality.
- **Task:** A single new addition to the codebase; this should typically map to an individual PR.

Task Naming:

- For clarity, the area of work should precede the title of the ticket in square brackets (e.E.G., [API], [DB], [FE]).

Task Requirements (by Type):

[Database]:

- Must include an ER diagram if new tables/relationships are added.
- Table name, columns, keys, constraints, and relations *must* be fully defined before refinement.

[API]:

- If an API ticket includes an update to request/response parameters, these changes *must* be documented as JSON in the ticket.
- Any feature flag and/or permission gating must be explicitly stated.
- Automated testing is implicit, but specific edge cases should be called out.

[FrontEnd]:

- A front-end task should map to an individual new component.
- If new API endpoints are used, request/response bodies should be listed as JSON in the ticket.
- Feature flag and permission gating must be explicitly called out.
- New unit tests must accompany new components; E2E tests should accompany new workflows.

Documentation

Summary

What level of documentation is needed and where should it be stored to make it valuable? Think about your current documentation and if it is useful. If not, discuss why. Also, discuss where documentation should be stored to create standards that make it easy to find and access.

Examples include product requirement documents, technical design documents, architectural diagrams and instructional README files. Each has a purpose and a different audience.

Key Questions & Discussion Prompts:

- What types of documentation do we value and agree to maintain?
- Where will our "single source of truth" live for different types of information?
    - *Example*: Product Requirements (e.g., Confluence, Jira Epics)?
    - *Example*: Technical Designs / Diagrams (e.g., Confluence, Miro, Readme)?
    - *Example*: Team Processes (like this) (e.g., Confluence, Guru, Readme)?
- What is our standard for a high-quality README.md in a repository?
- How do we handle and store documentation for architectural decisions (ADRs)?

Current Process

Discussion Notes

Agreements

All agreements made by the team need to be in this section. The agreements should be as detailed as possible.

Example Agreement: Documentation Location

> **Team Working Agreement (this doc):** Will live in our Confluence space.
>
> **Product Requirement Documents (PRDs):** Will live in Confluence, linked to the parent Epic in Jira.
>
> **Technical Design Documents:** Will live in Confluence, linked to the relevant Epic or Story.

Architectural Diagrams: Will be created in [Diagramming Tool] and embedded in the Confluence Technical Design Doc.

Code-Specific Instructions: Will be in the README.md file of the relevant repository.

Code Reviews

Summary

Code reviews are one of the strategies to create and maintain high-quality code. Every company has its own guidelines, but each team is responsible for the code they are creating. Discuss the current process and what is working well or not. If you have had quality issues, discuss how they could have been lessened with an agreed-upon process.

Key Questions & Discussion Prompts:

- What is the goal of our code review? (e.g., Find bugs, share knowledge, ensure standards, mentor).
- What is our expected turnaround time for a review request?
- How many approvals are needed to merge?
- What is the process? (e.g., Author posts PR, tag `@team-eng`, fills out template).
- Who merges the PR? (e.g., Author, Reviewer, CI/CD).
- What defines a "small" PR? Do we have a policy against large PRs?
- What is our policy on "LGTM" vs. detailed, constructive comments?
- When should we do an in-person or synchronous review instead of an async PR?

Current Process

Discussion Notes

Agreements

All agreements made by the team need to be in this section. The agreements should be as detailed as possible.

Example Agreement: Code Review Process

> **PR Template:** The PR template must be filled out, including a Loom video or screenshots/GIFs demonstrating proof of testing.
>
> **Approvals:** At least one (1) approval from an engineering teammate is required.
>
> **Process:** The author is responsible for resolving all comments.

Merging: Once approved, the author is responsible for merging their own PR.

Sizing: PRs should be small and ideally map to a single Task ticket. If a larger story is worked on, it should be broken into multiple PRs if possible.

On Call / Support

Summary

Any team that releases code is going to have a time when something goes wrong. An on-call / support process will define who is responsible for initially looking into issues and what their duties and responsibilities are. For this discussion, think about your team size, the tools you have, and what is expected from the on-call engineer. This includes rotation schedules, length of rotations, and escalation policies.

I also recommend using part of the on-call time for increasing code quality through tech debt or bug tickets. The on-call engineer can also be the first responder to questions from other teams, allowing the rest of the team to focus.

Key Questions & Discussion Prompts:

Rotation & Schedule:

- What is our rotation schedule? (e.g., One engineer rotating weekly, Monday to Monday?)
- Do we want a single on-call (L1) or a primary/secondary (L1/L2) system?

Escalation Policy:

- What is our backup/escalation path if the primary on-call is unavailable?
- *Example*: "The backup could be the on-call for next week".

Tools & Monitoring:

- What dashboards, logging, or tools do we need to effectively monitor our systems?
- How will we handle non-page interruptions? (e.g., Watching specific Slack channels).

Philosophy & Expectations:

- Do we agree that the on-call person *will not* be 100% focused on product work?
- What is the defined priority of their responsibilities?

On-Call Duties (The "Other" Work):

- Beyond pages, what is the on-call engineer responsible for?
- *Example Tasks*: Triage incoming bug tickets, be the first responder for questions from other teams, review noisy monitors, select old feature flags for archival, select a bug from the backlog to research and fix.

Current Process

- Pager Duty
    - Doesn't seem to be installable through my phone? Need to dig into this a little more
- Payer Retrieval Platform Pipelines
- Primary and secondary schedules
- 1 week rotation
- Primary moves to secondary after being primary
- Use incident.io for incidents
- Need more info on what an incident is because it seems like it is just high priority fixes that need visibility. Also when multiple teams need to be a part of the discovery/fix.
    - Payer Retrieval Incident Management
- Primary on-call is usually focused on support tickets during their week

- Support is current split between 2 boards
    - PPI - Payer Product Intake
        - There is an intake form that is supposed to be used, but seems like not always
    - PPPI - Payer Product Pod Issues
        - Linked to #support-payer Slack channel
        - Tickets created directly by CS

Discussion Notes

Agreements

All agreements made by the team need to be in this section. The agreements should be as detailed as possible.

Example Agreement: On-Call Duties & Responsibilities

Rotation:

- The on-call rotation will be one engineer at a time, switching every Monday at 10:00 AM.
- The backup/escalation will be the engineer scheduled for the following week.

Expectations:

- It is understood that the on-call engineer will not be 100% focused on product work. Their responsibilities are prioritized in the following order:

Priority of Responsibilities:

1. **Handle Pages:** Respond to any pages.
2. **Handle Interruptions:** Act as the first point of contact for external teams.
    - Watch the `#team-public-channel` for questions.

    - Triage any incoming escalations or tags to the `@team-eng`.
3. **Code Hygiene:** When not handling #1 or #2, work on code hygiene tasks.
    - Select one bug from the backlog to research and fix (or close if no longer relevant).
    - Select one LD (LaunchDarkly) flag for archival.
    - Review noisy monitors and recommend/make changes.
4. **Product Work:** If all other responsibilities are handled, the engineer can continue with their planned product work.

Additional Topics

Summary

This section is for topics that are important to your team but might not fit into the main categories.

Common Topics:

Tech Debt:

- How do we handle it? Is it included in standard workflows?
- Will the on-call engineer be handling it?
- How do you indicate a ticket is tech debt (e.g., Label, Epic, Fix Version)?

Tech Reviews / Knowledge Sharing:

- Do we want to do internal technical discussions?
- *Recommendation*: Have subject matter experts discuss patterns and practices. If you are doing something new, it is a good way to get the knowledge out.

Professional Development:

- Does the company support this? (e.g., time, expense reimbursements).
- *Recommendation*: Use part of the "no meeting days" as set-aside time for professional development.
- How will we track or share what we've learned?

Current Process

Discussion Notes

Agreements

All agreements made by the team need to be in this section. The agreements should be as detailed as possible.

Example Agreement: Tech Debt & Professional Development

Tech Debt:

- We will use the Tech Debt label in Jira for tracking.
- As part of the on-call duties, the on-call engineer will select one bug or tech debt ticket from the backlog to work on.
- We will hold a tech debt grooming session once per quarter to prioritize the backlog.

Professional Development:

- We will block 4 hours every other Friday for "Focus & Dev Time."
- This time can be used for professional development, finishing up work or working on tech debt.
- Team members will share what they worked on/learned in a shared spreadsheet to benefit the rest of the team.

Topic Template

Summary

Current Process

Discussion Notes

Agreements